
Nexidia Real-Time Grid Programmer's Guide

Version 12.5.1

Table of Contents

Copyright

Programmer's Introduction

Overview

Basic Concepts

Speech Search

Device Acoustics

Speakers

Workflows, Workflows and Enlighten

Real-Time Automatic Speech Recognition

Enlighten

Search Term Validation

Integrating with Real-Time Grid

Rules Configuration

Event Consumption

HTTP Speech Analysis Interface

Design Notes

Documentation Conventions

Error Responses

Global Error Responses

Operations

Get System Info

Add Workflow

Delete Workflow

Get Workflow Inventory

Find Workflow

Get Workflow Info

Get Workflow Manifest

Search Term Validation

Set Configuration

Get Configuration

- Set Append Metadata
- Get Append Metadata
- Set Agent Information
- Get Agent Information
- Get System Topology
- Set Air Information
- Get Air Information
- Messaging Interface
 - Destinations
 - Standard Message Properties
 - Call Event Messages
 - CallStart
 - CallPause
 - CallResume
 - CallEnd
 - Telephony Proxy metadata filtering
 - Event Notification Message
 - Enlighten Update Message
 - Transcript Message
- Nexdia Interaction Analytics (NIA) Integration
 - Notification Payload
 - Example Notification Payload
- References

Copyright

PROPRIETARY AND CONFIDENTIAL INFORMATION

Information herein is proprietary information and trade secrets of NICE Ltd. and/or its affiliated companies (Affiliates). This document and the information herein is the exclusive property of NICE and its Affiliates and shall not be disclosed, in whole or in part, to any third party or utilized for any purpose other than the express purpose for which it has been provided.

IMPORTANT NOTICE

Subject always to any existing terms and conditions agreed between you and NICE or any Affiliate with respect to the products which are the subject matter of this document, neither NICE nor any of its Affiliates shall bear any responsibility or liability to a client or to any person or entity with respect to liability, loss or damage caused or alleged to be caused directly or indirectly by any product supplied or any reliance placed on the content of this document. This includes, but is not limited to, any interruption of service, loss of business or anticipatory profits or consequential

damage resulting from the use or operation of any products supplied or the content of this document. Information in this document is subject to change without notice and does not represent a commitment on the part of NICE or any Affiliate.

All information included in this document, such as text, graphics, photos, logos and images, is the exclusive property of NICE or in Affiliate and is protected by United States and international copyright laws. Permission is granted to use, view and photocopy (or print) materials from this document only in connection with the products to which this document relates and subject to the terms of license applicable to such products. Any other use, copying, distribution, retransmission or modification of the information in this document without the express prior written permission of NICE or an Affiliate is strictly prohibited. In the event of any permitted copying, redistribution or publication of copyrighted material, no changes in, or deletion of, author attribution, trademark legend or copyright notice shall be made.

Products supplied may be protected by one or more of the US patents listed at www.nice.com/Patents. For the full list of trademarks of NICE and its Affiliates, visit www.nice.com/Nice-Trademarks. All other marks used are the property of their respective proprietors.

All contents of this document are: Copyright © 2025 NICE Ltd. All rights reserved.

For assistance, contact your local supplier or nearest NICE Customer Service Center:

EMEA Region (Europe, Middle East, Africa) Tel: +972-9-775-3800 Fax: +972-9-775-3000 email: support@nice.com	APAC Region (Asia/Pacific) Tel: +852-8338-9818 Fax: +852-2802-1800 email: support.apac@nice.com	North America Tel: 1-800-663-5601 Fax: +201-356-2197 email: na_sales@nice.com	International Headquarters-Israel Tel: +972-9-775-3100 Fax: +972-9-775-3070 email: info@nice.com
The Americas Region (North, Central, South America) Tel: 1-800-6423-611 Fax: +720-264-4012 email: support.americas@nice.com	Israel Tel: 09-775-3333 Fax: 09-775-3000 email: support@nice.com	France Tel: +33-(0)1-41-38-5000 Fax: +33-(0)1-41-38-5001	Hong-Kong Tel: +852-2598-3838 Fax: +852-2802-1800
United Kingdom Tel: +44-8707-22-4000 Fax: +44-8707-22-4500	Germany Tel: +49-(0)-69-97177-0 Fax: +49-(0)-69-97177-200		

NICE invites you to join the NICE User Group (NUG).

Visit the NUG Website at www.niceusergroup.org; and follow the instructions.

All queries, comments, and suggestions are welcome! Please email: nicebooks@nice.com.

For more information about NICE, visit www.nice.com

Programmer's Introduction

Overview

Nexidia Real-Time Grid (RTG) performs real-time Automatic Speech Recognition (ASR) analysis on telephone calls in real time. Real-Time Grid is not a stand-alone application; rather, it is a service-based application framework that provides speech analysis capabilities for a client application. This document describes the interactions between Real-Time Grid and a client application.

Basic Concepts

Speech Search

RTG's real-time ASR search engine searches speech in real-time, provides a transcript of speech in the audio, and identifies specified words and phrases as they are spoken.

Two factors each contribute significantly to the quality of the real-time ASR search:

- the acoustics of the device used to record the speech, and
- the linguistic characteristics of the speakers.

Device Acoustics

The device used to record speech shapes the waveform in significant ways. Microphones from different manufacturers have different frequency responses, which can cause significant alterations in how a waveform is recorded. Some devices may introduce distortion, such as clipping or reverberation.

In addition, the circumstances under which the speech was recorded can also alter the waveform. Audio might include background noise, such as other people speaking faintly, music, or machinery. Telephone interactions often include dial tones or tones from pressing the telephone buttons.

Speech recorded in a quiet studio with a broadcast-quality microphone and other professional equipment is going to produce a different waveform from the same speech recorded through a telephone in a Laundromat.

Speakers

The way people speak varies at least as much as the acoustics of a given set of devices. People in broadcast studios tend to speak differently from people in Laundromats shouting into telephones; rehearsed speech has different rhythms and intonation from spontaneous speech.

Similarly, the same words will produce a different waveform when spoken by a middle-aged man from Portland, Maine; a young woman from Huntsville, Alabama; or a child from Los Angeles, California. Even the same speaker saying the same thing will produce a different waveform when they say it with a head cold.

Workflows, Workflows and Enlighten

A Workflow is a ZIP file that is loaded into RTG at runtime. A Model contains one or more Workflows.

A Workflow performs Automatic Speech Recognition (ASR) for a single language and localization, such as North American English or Spanish. A Workflow also contains one or more Enlighten models. A Workflow includes a Dictionary of allowable words.

An Enlighten model performs sentiment analysis on a specific sentiment of interest to the Agent. Examples might include 'Show Loyalty,' 'Show Empathy,' 'Take Ownership,' as positive sentiment, or 'Rapid Speech' as negative sentiment. An Enlighten model is bound to its containing Workflow.

Real-Time Automatic Speech Recognition

Most Nexidia products search recorded audio by creating a phonetic index after the audio stream is complete. In contrast, Real-Time Grid uses ASR to search audio in real time as the words are being spoken.

To use RTG for real-time ASR, specify a set of terms for which to search as well as a set of optional parameters that refine the search algorithm. A search term can be any combination of dictionary words or names (such as Albert Einstein, London Bridge, or Weekaugen, Missouri). The search terms must be included in the Workflow Dictionary or the term will not be allowed for searching.

Search terms are organized into categories called phrase expressions. Search terms within a phrase expression can be constrained to match only within a certain time span of another word or phrase.

Phrase expressions are organized into rule definitions. Each rule definition expresses a set of conditions that, when met, cause RTG to publish an event notification message. An event can be either discrete or continuous. A discrete event occurs at a single point in time when a phrase of interest is detected. Discrete events result in a single primary notification message. A continuous event occurs over a span of time. Examples of continuous events include "negated" phrase expressions (i.e. a span of time in which a particular phrase is not uttered) and periods of interesting Enlighten scores. Continuous events can result in multiple notification messages sent periodically over the duration of the event. The first notification send for a continuous event is considered primary and subsequent messages sent during the same event are secondary.

Event notification messages include several pieces of information:

- a reference to the call and the party on the call
- a reference to the rule definition associated with the notification
- a time offset from the start of the call when the match was established
- a payload of arbitrary client application data

Enlighten

Enlighten evaluates the likelihood that a particular audio stream matches a criterion. The criterion evaluated depends on the Enlighten model being applied to the audio. For example, an Enlighten model might use the content of a phone call to predict how the customer would respond to a satisfaction survey after the call. This kind of analysis is called sentiment detection.

The output of an Enlighten model is a score 0-1 that is updated periodically over the duration of the audio stream. The interpretation of the score depends on the model being used. For example, with a sentiment detection model higher scores could indicate increasing likelihood of a negative survey response. Enlighten scores are used in two ways: First, the system broadcasts periodic update messages with current scores; second, rule definitions can incorporate Enlighten scores into their conditional triggers.

Each Enlighten model represents a behavior which may be positive or negative. A score represents the likelihood of a behavior which could be good or bad. A high Empathy score, for example, would be good while a high Churn score would be bad. It is the purview of the consuming application how the scores are used for display.

Enlighten scores are normalized as follows:

- Scores above 0.9 indicate the behavior is strongly present.
- Scores between 0.7 and 0.9 indicate the model is starting to occur, with perhaps milder expressions of the behavior (for example: "I disagree" vs. "I demand to speak to your manager").
- Scores below 0.7 indicate the behavior is not occurring or only being weakly/sporadically expressed.

Each score also has an accompanying 'isValid' boolean. If isValid is 'true' it means enough conversation had taken place when the score was emitted that the score can be considered valid. If isValid is 'false' insufficient conversation has occurred to produce a valid score.

An Enlighten model may be designed to operate on the Agent audio stream, the Other audio stream, or Any, meaning either stream. A model designed to monitor the Agent stream would always have isValid set to false if applied to the Other stream.

Search Term Validation

Each Workflow has its own Dictionary of valid search terms. To help end users in constructing valid terms and phrases, RTG provides an API with which client applications can validate search terms before scanning for them.

If a search term is invalid, RTG will return an error, including an array of characters that cause the term to be invalid, if they can be identified.

If the search term is valid, RTG will return success.

Integrating with Real-Time Grid

Real-Time Grid is an application framework, not a stand-alone application. Therefore, client applications must fulfill a minimum two categories of functional requirements to produce business value - Rules Configuration and Event Consumption.

Rules Configuration

RTG requires configuration data to tell it how to analyze calls. Configuration information includes what Workflows and Enlighten models to apply, what terms to search for, and what phrase expression operators to evaluate. The client application provides this information to RTG via an HTTP interface.

Event Consumption

Real-Time Grid publishes the results of its ASR and Enlighten analyses via an asynchronous messaging system. For the results to be useful, the client application must receive these messages and incorporate them into the application's business logic.

HTTP Speech Analysis Interface

Design Notes

- The HTTP API design for real-time ASR and Enlighten analysis largely adopts the style known as "hypertext as the engine of application state" (HATEOAS; Fielding, 2000).
- Internal URI structure is not specified, with two exceptions:
 - URIs do not have query or fragment components unless explicitly noted.
 - URI query components conform to the encoding rules for "application/x-www-form-urlencoded" URLs (W3C, 1999).
- *Inventory* means a list of related resources of a given type.
- *Info* means an entity describing aspects of a specific resource.
- All JSON request and response entities are content type "application/vnd.nexidia.realtimelog+json" unless otherwise specified.

Documentation Conventions

Because URI structure is not part of the public spec, this document uses "dummy" URIs (e.g. "<http://example/media1>" (<http://example/media1>")) in place of real URIs. In contrast, URI query parameters are part of the public spec for some resource types.

Error Responses

HTTP error responses include several types of information:

- The request ID is a string identifying the client application's HTTP request. It can be used to correlate error events with Real-Time Grid log files.

https://wem-dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework/NXQC_12.5/Grid_Documentati...

Updated: Thu, 30 Apr 2026 18:13:28 GMT

- The error code is a well-known string signifying the type of error. The error code is intended for automated processing, but it is human-readable as well.
- The error resource URI indicates a resource relevant to the error. In most cases, the error resource is the same as the HTTP request URI. But some errors point to a different resource; for example, to indicate a conflict.
- The error message is a human-readable string describing the error. Error messages are intended for administrator or technical support users, so client applications should not present RTG error messages directly to end users. Error message content can change from release to release, so client applications should base error handling logic on the error code, not the error message.

Error information is returned in HTTP headers and, when applicable, in HTTP response entities. The following HTTP headers are available in error responses:

- `x-nx-request-id`—Contains the request ID. This header is available in all Real-Time Grid HTTP responses, but it is most useful in error situations.
- `x-nx-error-code`—Contains the error code.
- `x-nx-error-resource`—Contains the error resource URI. If the client’s HTTP request indicated an `Accept` header value of `"application/vnd.nexidia.realtimeweb+json"`, error responses include an entity with the structure shown below.

TIP

The error entity is not included in responses to HEAD requests.

```
{
  "code" : "UnkownResource",
  "message" : "Media with the same external media ID already exists.",
  "resourceUri" : "http://example/mediainventory",
  "requestId" : "7c83aba7-4bc1-4eb4-b0aa-1386df67eaff"
}
```

JSON

Global Error Responses

Error	Code	Message	Resource URI
500 Internal Server Error	InternalServerError	Internal error occurred while processing request.	The request URI.
400 Bad Request	BadRequest	Requested URL or input entity was malformed.	The request URI.
413 Request Entity Too Large	RequestEntityTooLarge	Input entity is too large for the server to accept.	The request URI.
503 Service Unavailable	ServiceUnavailable	The server is currently unable to handle the request due to a temporary overloading or maintenance of the server.	The Request URI.

Error	Code	Message	Resource URI
503 Service Unavailable	InitializationInProgress	Request cannot be completed because the server is being initialized.	The request URI.
503 Service Unavailable	ShutdownInProgress	Request cannot be completed because the server is being shut down.	The request URI.

Operations

Get System Info

This operation provides the starting point for HATEOAS interactions.

The version member should be treated as an opaque string with no machine-parsable format.

Request

The system URI must be provided to the client application out-of-band.

```
GET [System URI] HTTP/1.1
Accept: application/vnd.nexidia.realtimegrid+json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
{
  "version" : "1.0.0.0 (123456)"
  "configurationUri" : "http://example/configuration",
  "workflowInventoryUri" : "http://example/workflowInventory",
  "agentUri" : "http://example/agent",
  "appendMetadataUri" : "http://example/appendMetadata",
  "searchTermValidationUri" : "http://example/searchTermValidation"
}
```

Add Workflow

Installs a new workflow to the RTG instance. The request entity is an archive file in ZIP format. The manifest.xml file should be no more than one directory level below the archive's root.

Request: Upload a Zip file

Workflow Inventory URI is provided by System Info.

```
POST [Workflow Inventory URI] HTTP/1.1
Accept: application/vnd.nexidia.realtimeweb+json
Content-Type: application/zip
[Request entity is the zipped Workflow.]
```

Response: Success

```
HTTP/1.1 201 Created
Location: [Specific Workflow URI]
Content-Type: application/vnd.nexidia.realtimeweb+json
{
  "guid" : "966da303-bde8-4d9d-83e4-d2c49af6ae45",
  "name" : "NA English 9.3.0.59720",
  "tag" : "Default 9.3.8 (North American English)",
  "languageId" : 0,
  "languageName" : "North American English",
  "description" : "Real Time Grid ingest test package",
  "minSupportedWorkbenchVersion" : "10.6",
  "isEnlightenIncluded" : true,
  "formatVersion" : "1",
  "manifestUri" : "http://example/workflowmanifest",
  "searchTermValidationUri" : "http://example/searchtermvalidation"
}
```

Response: Conflicting Workflow

```
HTTP/1.1 409 Conflict
Content-Type: application/vnd.nexidia.realtimeweb+json
{
  "code" : "ConflictingWorkflow",
  "message" : "A Workflow with the same GUID is already installed.",
  "resourceUri" : "[The conflicting Workflow info URI]"
}
```

Response: Invalid Workflow

```
HTTP/1.1 400 Bad Request
Content-Type: application/vnd.nexidia.realtimeweb+json
{
  "code" : "InvalidWorkflow",
  "message" : "The submitted Workflow is invalid.",
  "resourceUri" : "[The request URI]"
}
```

Delete Workflow

Removes an installed Workflow from the RTG instance.

The request will fail if the current configuration references the Workflow to be deleted. If this happens, update the configuration to remove the usage of the model to be deleted, then invoke Delete Workflow again.

Request

Workflow URI provided by Workflow Inventory <https://docs.nexidia.com/nexidia-analytics-and-data-exchange-framework/NXQC-12.5/Grid-Documentati...>

Updated: Thu, 30 Apr 2026 18:13:28 GMT

```
DELETE [Workflow URI] HTTP/1.1
```

Response: Success

```
HTTP/1.1 204 No Content
```

Response: Workflow in Use

```
HTTP/1.1 403 Forbidden
Content-Type: application/vnd.nexidia.realtimegrid+json
{
  "code" : "WorkflowInUse",
  "message" : "Workflow is referenced by the current configuration.",
  "resourceUri" : "[The request URI]"
}
```

Get Workflow Inventory

Request

Workflow Inventory URI provided by System Info.

```
GET [Workflow Inventory URI] HTTP/1.1
Accept: application/vnd.nexidia.realtimegrid+json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
{
  "workflowUris" : [
    "http://example/workflow1",
    "http://example/workflow2"
  ]
}
```

Find Workflow

Returns the Workflow URI of a Workflow with the specified GUID.

TIP

Some HTTP client frameworks can automatically follow redirection responses. If the client application should cache the returned URI, automatic redirection following should be disabled.

Request

```
GET [Workflow Inventory URI]?workflowGuid=<workflowGuid> HTTP/1.1
Accept: application/vnd.nexidia.realtimegrid+json
```

Response: Success

```
HTTP/1.1 200 OK
Location: [Specific Workflow URI]
```

Response: Unknown Workflow

```
HTTP/1.1 404 Not Found
Content-Type: application/vnd.nexidia.realtimegrid+json
{
  "code" : "UnknownResource",
  "message" : "The specified Workflow is not installed.",
  "resourceUri" : "[The request URI]"
}
```

Get Workflow Info

Request

Workflow URI provided by Workflow Inventory.

```
GET [Workflow URI] HTTP/1.1
Accept: application/vnd.nexidia.realtimegrid+json
```

Response: Success

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
{
  "guid" : "966da303-bde8-4d9d-83e4-d2c49af6ae45",
  "name", "NA English 9.3.0.59720",
  "tag":"Default 9.3.8 (North American English)",
  "languageId" : 0,
  "languageName" : "North American English",
  "description":"Real Time Grid ingest test package",
  "minSupportedWorkbenchVersion":"10.6",
  "isEnlightenIncluded":true,
  "formatVersion":"1",
  "manifestUri" : "http://example/workflowmanifest",
  "searchTermValidationUri" : "http://example/searchtermvalidation"
}
```

Response: Unknown Workflow

```
HTTP/1.1 404 Not Found
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Get Workflow Manifest

This operation, unlike most other Real-Time Grid operations, returns MIME type application/xml.

Request

Workflow Manifest URI provided by Workflow Info.

```
GET [Workflow Manifest URI] HTTP/1.1
Accept: application/xml
```

Response: Success

```
Content-Type: application/xml
[entity contains Workflow manifest]
```

The XML namespace of the response entity is that of the workflow manifest.

Response: Unknown Workflow

No entity is returned.

```
HTTP/1.1 404 Not Found
```

Search Term Validation

This operation can be used to validate that a search term is allowable for a Workflow.

Entity Members

Entity Member	Description
queryString	If normalization is requested, this member contains the normalized term. Otherwise, it contains the original search term.
invalidWords	Empty if all words in phrase are words found in the lexicon for the relevant Workflow; otherwise contains a list of objects for each invalid word. Object contains the fields ['invalidWord', 'error', 'offset'].

Request

Search Term Validation URI provided by Workflow Info.

```
GET [Search Term Validation URI]?queryString=<searchTerm> HTTP/1.1
Accept: application/vnd.nexidia.realtimegrid+json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
https://wem-dochub.mindtouch.us/Nexidia Analytics and Data Exchange Framework/NXQC 12.5/Grid Documentati...
```

Example response entity for a valid search term:

```
{
  "queryString" : "Search for this",
  "invalidWords" : []
}
```

JSON

Example response entity for an invalid search term:

```
{
  "queryString" : "Hello world yessir",
  "invalidWords" : [
    {
      "invalidWord": "yessir",
      "error": "OUTOFLEXICON",
      "offset": 10
    }
  ]
}
```

JSON

- A response of 200 (OK) does not imply that the term is valid for searching, only that validation was executed successfully.
- Bad character offsets conform to the spec used by the Workbench `phraseBuilderValidatePhrase()` function.

Response: Unknown Workflow

```
HTTP/1.1 404 Not Found
{
  "code": "UnknownWorkflow",
  "message": "The specified Workflow is not installed.",
  "resourceUri": "[The request URI.]"
}
```

Set Configuration

Updates configuration for the RTG. Configuration changes do not apply to calls in progress. Configuration changes may take several seconds to apply to all services in a distributed deployment. Send an empty message body to reset the configuration to its initial state. The **Content-Type** header must be set.

Entity Members

All members are required unless otherwise specified.

Entity Member	Member	Member/ Operator	Definition/Value
configurationId			An optional string used to identify this configuration. If set, this value is emitted in Event Notification messages to correlate notifications with the configuration revision used to emit them. The value must be 64 characters or less, but otherwise the format is client-specified.
workflowFilters			An array of filters used to assign workflows to calls. The first filter to match defines the Workflow used for the call, even if subsequent filters would match as well. At least one filter must be defined.
	workflowGuid		GUID identifying the Workflow to be used if this filter matches. If this member is omitted, the call will not be scanned on a match.
		operator	How to evaluate individual filters. If this member is omitted, the associated Workflow and Enlighten models will always be applied, and no subsequent filters will be evaluated. This can be used at the end of a workflowFilters array to define a "default" behavior. Two values are supported: All:Each filter must evaluate to true. Any: At least one filter must evaluate to true.

Entity Member	Member	Member/ Operator	Definition/Value
		filters	Array of filters to be evaluated. May be omitted if operator is omitted. key: Name of the metadata field to evaluate. Evaluation of key names is case-sensitive. operator: How to evaluate the metadata field. Two operators are supported: * IsIn: Value must be present in the list. * IsNotIn: Value must not be present in the list. values: Array of strings to evaluate against the metadata value. Evaluations are not case-sensitive.
eventDefinitions			An array of events to detect in calls. Each rule definition expresses a set of conditions under which Real-Time Grid will send a notification message. This member is optional, and is normally omitted in an environment using only Enlighten score updates.
	eventId		Positive integer uniquely identifying the Event definition within this configuration.
	workflowGuid		Required field; GUID identifying the Workflow for this rule definition. Use the Get Workflow Info operation to inspect available Workflow GUID values.

Entity Member	Member	Member/ Operator	Definition/Value
	maxNotificationCount		Optional field. The default is infinity. The maximum number of times a primary notification message for this event definition may be sent for a single call. Once this limit is reached, no further primary notification messages will be sent during this call for this type of event.
	secondaryTimeBasisMs		Applies only to Continuous Events. Required for Enlighten rule or omitted phrase rule. The number of milliseconds between successive notifications of a continuous event. Event notification messages will be repeatedly sent for the duration of the event so long as the maxSecondaryNotificationCount value is not exceeded.
	maxSecondaryNotificationCount		Applies only to Continuous Events. The default is infinity. The maximum number of times a secondary notification message may be sent during a continuous event. Once this limit is reached, no further secondary notification messages will be sent during this event. The count limit is enforced for each occurrence of a continuous event. So if one event ends and the same type of continuous event is detected later, the count begins again. This field is optional with a default value of zero.
	cooldownMs		Applies only to discrete Events. Minimum number of milliseconds that must pass after an event detection before it can be detected again. This prevents sending multiple notifications in a short period, which is especially likely when the operator value is set to "Any." If not specified, defaults to "never cool down."
https://wem-dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework_VXQC_12.5/Grid_Documentati...			

Updated: Thu, 30 Apr 2026 18:13:28 GMT

Entity Member	Member	Member/ Operator	Definition/Value
	metadataFilters		Optional structure defining call metadata conditions that must be true to scan the call.
		operator	How to evaluate the individual filter conditions. Two values are supported: Any: will be scanned if at least one filter evaluate to true. All: will be scanned only if all filters evaluate to true.
		filters	Array of filter conditions to apply to metadata. key: Name of the metadata field to evaluate. Evaluation of key names is case-sensitive. operator: How to evaluate the metadata field. Two operators are supported: * IsIn: Value must be present in the list. * IsNotIn: Value must not be present in the list. values: Array of strings to evaluate against the metadata value. Evaluations are not case-sensitive.
	notificationPayload		Optional structure containing client-specified name-value pairs. Real-Time Grid emits these members in event notification messages.
	rule		Expresses a set of conditions evaluating to true or false.
		ruleId	Positive integer uniquely identifying the Rule definition within this configuration.

Entity Member	Member	Member/ Operator	Definition/Value
		enlightenModelId	GUID identifying the model from which to derive scores.
		aggregationType	Describes how rules two rules within this rule will be evaluated together. Aggregate rules may be nested. Required only for aggregate rules. And: Both leaf rules must evaluate to true. Or: Either leaf rule may evaluate to true to cause the aggregate to evaluate to true. Within: Both leaf rules must evaluate to true within the timeBasisMs to cause the aggregate rule to evaluate to true. NotAny: No leaf rule may evaluate to true for the aggregate rule to evaluate to true. NotAll: At least one leaf rule must evaluate to false for the aggregate to evaluate to true.
		threshold	The threshold with which to compare the Enlighten score. Required for an Enlighten event.
		direction	The direction with which the rule evaluates the score's relationship to the threshold. Required for an Enlighten event. GreaterThan: The Enlighten score must be greater than the threshold. LessThan: The enlighten score must be less than the threshold.

Entity Member	Member	Member/ Operator	Definition/Value
		minDurationMs	The minimum number of milliseconds during which the Enlighten score must be above the threshold value to trigger a notification message.
		phraseExpression	An optional set of terms to evaluate for the event. If this member is omitted, the enlighten member must be present.
		stream	Identifies the relevant audio stream on the call. Required if phraseExpression is present; optional otherwise. Three values are supported: Agent: Evaluate the event against the monitored agent's audio stream. Other: Evaluate the event against the non-agent audio stream. In many call centers, this stream contains customer audio. In transfer or conference call scenarios, call center agents may appear in this stream. Either: Evaluate the event against the Agent and Other streams. Note that the streams are evaluated independently, not as a single merged stream.
		timeBasisOperator	Together with the timeBasisMs member, describes time relationships under which a notification message will be sent. Three values are supported: Anywhere: Utterances throughout the call are considered. First: Only utterances in the initial section of the call are considered. NotFirst: Only utterances after the initial section of the call are considered.
https://wem-dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework/NXQC_12.5/Grid_Documentati...			

Entity Member	Member	Member/ Operator	Definition/Value
		timeBasisMs	The number of milliseconds to use for the selected timeBasisOperator . Required unless timeBasisOperator is "Anywhere."
		triggeredBy	If non-zero, the id of a rule that will cause this rule to evaluate.
		minHitCount	The minimum number of utterances required to contribute towards the phrase operator. This field is optional, with a default value of 1. Values greater than one are invalid if the phrase operator is "NotAll" or "NotAny."

Request

Configuration URI provided by System Info.

```
PUT [Configuration URI] HTTP/1.1
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Sample:

```

{
  "configurationId": "77",
  "workflowFilters": [
    {
      "workflowGuid": "e78f7d08-ae84-4248-ab99-7c20b7181d77",
      "operator": "Any",
      "filters": [
        {
          "key": "skillGroupId",
          "operator": "IsIn",
          "values": ["1", "2", "3"]
        }
      ]
    }, {
      "workflowGuid": "95e216b3-64e6-412c-915c-b75599943c86",
      "operator": "All",
      "filters": [
        {
          "key": "skillGroupId",
          "operator": "IsIn",
          "values": ["8", "9", "11"]
        }, {
          "key": "agentAcidLoginId",
          "operator": "IsNotIn",
          "values": ["9354"]
        }
      ]
    }
  ],
  "eventDefinitions": [
    {
      "event": {
        "eventId": 88,
        "workflowGuid": "55555555-5555-5555-5555-555555555555",
        "maxNotificationCount": 0,
        "secondaryTimeBasisMs": 0,
        "maxSecondaryNotificationCount": 0,
        "cooldownMs": 0,
        "metadataFilters": {
          "operator": "Any",
          "filters": [
            {
              "key": "MetadataKey",
              "operator": "IsIn",
              "values": [
                "MetadataValue1",
                "MetadataValue2",
                "MetadataValue3"
              ]
            }
          ]
        }
      },
      "notificationPayload": {
        "articleId": 42,
        "articleUrl": "http://customer:8080/articles/42.html",
        "dialogType": "Reminder"
      },
      "rule": {
        "ruleId": 1,
        "phraseExpression": "cancel my account",
        "stream": "Either",
        "timeBasisOperator": "First"
      }
    }
  ]
}

```

```

"timeBasisMs": 0,
"triggeredBy": [3, 2],
"minHitCount": 1
}
},
{
"event": {
"eventId": 55,
"workflowGuid": "55555555-5555-5555-5555-555555555555",
"maxNotificationCount": 0,
"secondaryTimeBasisMs": 0,
"maxSecondaryNotificationCount": 0,
"cooldownMs": 0,
"metadataFilters": {
"operator": "All",
"filters": [
{
"key": "MetadataKey",
"operator": "IsNotIn",
"values": [
"MetadataValue1",
"MetadataValue2",
"MetadataValue3"
]
}
]
},
},
"notificationPayload": {
"articleId": 42,
"articleUrl": "http://customer:8080/articles/42.html",
"dialogType": "Reminder"
},
"rule": {
"ruleId": 2,
"enlightenModelId": "00000000-0000-0000-0000-000000000000",
"threshold": 0.75,
"direction": "GreaterThan",
"minDurationMs": 2000,
"timeBasisMs": 0,
"timeBasisOperator": "Anywhere",
"triggeredBy": [],
"minHitCount": 1
}
},
{
"event": {
"eventId": 33,
"workflowGuid": "55555555-5555-5555-5555-555555555555",
"maxNotificationCount": 0,
"secondaryTimeBasisMs": 0,
"maxSecondaryNotificationCount": 0,
"cooldownMs": 0,
"metadataFilters": {
"operator": "Any",
"filters": [
{
"key": "MetadataKey",
"operator": "IsIn",
"values": [
"MetadataValue1"

```

```

"MetadataValue2",
"MetadataValue3"
]
}
]
},
"notificationPayload": {
"articleId": 42,
"articleUrl": "http://customer:8080/articles/42.html",
"dialogType": "Reminder"
},
"rule": {
"ruleId": 3,
"aggregationType": "Or",
"timeBasisMs": 10000,
"metadataFilters": {
"operator": "Any",
"filters": [
{
"key": "MetadataKey",
"operator": "IsIn",
"values": [
"MetadataValue1",
"MetadataValue2",
"MetadataValue3"
]
}
]
}
},
"rule": [
{
"ruleId": 4,
"phraseExpression": "please understand",
"stream": "Agent",
"timeBasisOperator": "Anywhere",
"timeBasisMs": 0,
"triggeredBy": [1],
"minHitCount": 1
},
{
"ruleId": 5,
"enlightenModelId": "00000001-0000-0000-0000-000000000000",
"threshold": 0.8,
"direction": "GreaterThan",
"minDurationMs": 5000,
"timeBasisMs": 0,
"timeBasisOperator": "Anywhere",
"maxNotificationCount": 0,
"secondaryTimeBasisMs": 0,
"maxSecondaryNotificationCount": 0,
"cooldownMs": 0,
"triggeredBy": [1, 2],
"minHitCount": 1
}
]
}
},
{
"event": {
"eventId": 11,
"workflowGuid": "55555555-5555-5555-5555-555555555555"
}
}

```



```

"maxNotificationCount": 0,
"secondaryTimeBasisMs": 0,
"maxSecondaryNotificationCount": 0,
"cooldownMs": 0,
"metadataFilters": {
  "operator": "Any",
  "filters": [
    {
      "key": "MetadataKey",
      "operator": "IsIn",
      "values": [
        "MetadataValue1",
        "MetadataValue2",
        "MetadataValue3"
      ]
    }
  ],
},
"notificationPayload": {
  "articleId": 42,
  "articleUrl": "http://customer:8080/articles/42.html",
  "dialogType": "Reminder"
},
"rule": {
  "ruleId": 6,
  "aggregationType": "And",
  "timeBasisMs": 10000,
  "rule": [
    {
      "aggregationType": "And",
      "timeBasisMs": 10000,
      "ruleId": 7,
      "rule": [
        {
          "ruleId": 8,
          "phraseExpression": "welcome",
          "stream": "Either",
          "timeBasisOperator": "Anywhere",
          "timeBasisMs": 0,
          "triggeredBy": [1, 2],
          "minHitCount": 1
        }
      ],
    },
    {
      "ruleId": 9,
      "phraseExpression": "acme incorporated",
      "stream": "Either",
      "timeBasisOperator": "First",
      "timeBasisMs": 0,
      "triggeredBy": [],
      "minHitCount": 1
    }
  ],
},
{
  "ruleId": 10,
  "enlightenModelId": "00000001-0000-0000-0000-000000000000",
  "threshold": 0.8,
  "direction": "GreaterThan",
  "minDurationMs": 5000,
  "timeBasisMs": 0,
  "timeBasisOperator": "Anywhere",
  "triggeredBy": []
}

```

```
"minHitCount": 1
}
]
}
}
}
]
}
```

Response: Success

```
HTTP/1.1 204 No Content
```

Response: Invalid

```
HTTP/1.1 400 Bad Request
Content-Type: application/vnd.nexidia.realtimegrid+json
{
  "code": "InvalidConfiguration",
  "message": "[The reason that the configuration is invalid.]",
  "resourceUri": "[The request URI.]"
}
```

- Use the Validate Search Term operation to check phrases before adding them to a configuration.
- Sending an empty message body resets the configuration to its initial state.

Get Configuration

Retrieves the current configuration data. See Set Configuration for a description of the returned entity.

Request

Configuration URI provided by System Info.

```
GET [Configuration URI] HTTP/1.1
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: Configuration Is Set

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
```

See Set Configuration for the entity description.

Response: No Configuration Set

```
HTTP/1.1 204 No Content
```

Set Append Metadata

Updates the metadata for call events. The metadata is appended to the existing metadata for the call. The **Content-Type** header must be set to application/vnd.nexidia.realtimegrid+json.

Entity Member	Member				Definition
appendRule					List of rules to check against in order to add metadata to the call event.
	newKey				The key of the new metadata field to add to the call event.
	value				The value of the new metadata field to add to the call event.
	conditionTree				Group of conditions that determine if the metadata is added to the call event.
		operator			Determines if the conditions are evaluated individually or as a group. Supported values are OR if only one condition needs to evaluate to true, or AND if all conditions need to evaluate to true.
		operands			List of conditions to determine if the metadata is added to the call event.
			operator		Determines if the incoming metadata values are evaluated individually or as a group. Supported values are OR if only one value needs to evaluate to true, or AND if all values need to evaluate to true.
			operands		List of conditions applied to the incoming call event's metadata to determine if the new metadata should be added to the call event.
				key	Name of the incoming call event field to evaluate. Key name is case-sensitive.
				patterns	List of strings to evaluate against the incoming call event's field value described by the key property. Patterns are not case-sensitive.

Request

Append Metadata URI provided by System Info.
https://wem-dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework/NXQC_12.5/Grid_Documentati...
Updated: Thu, 30 Apr 2026 18:13:28 GMT

```
PUT [Append Metadata URI] HTTP/1.1
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: Success

```
HTTP/1.1 204 No Content
```

Response: Invalid Append Metadata

```
HTTP/1.1 500 Server Error
Content-Type: text/plain
Invalid Append Metadata File
```

Sample:

```
{
  "appendRules": [{
    "newKey": "LOB",
    "conditionTree": {
      "operator": "OR",
      "operands": [{
        "operator": "OR",
        "operands": [{
          "key": "agentPhoneExtension",
          "patterns": ["5020"]
        }]
      }]
    },
    "value": "Business One"
  }, {
    "newKey": "LOB",
    "conditionTree": {
      "operator": "OR",
      "operands": [{
        "operator": "OR",
        "operands": [{
          "key": "agentPhoneExtension",
          "patterns": ["501.*", "120.*", "121.*", "400.*"]
        }]
      }]
    },
    "value": "Business Two"
  }, {
    "newKey": "LOB",
    "conditionTree": {
      "operator": "OR",
      "operands": [{
        "operator": "OR",
        "operands": [{
          "key": "agentPhoneExtension",
          "patterns": ["500.*"]
        }]
      }]
    },
    "value": "New Line of Business"
  }
]
}
```

Get Append Metadata

Retrieves the current append metadata information. See Set Append Metadata for a description of the returned entity.

Request

Append Metadata URI provided by System Info.

```
GET [AppendMetadata URI] HTTP/1.1
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: Append Metadata Is Set

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: No Append Metadata Set

```
HTTP/1.1 204 No Content
```

Set Agent Information

Updates the agents in RTG to subscribe to the telephony events. The **Content-Type** header must be set to application/vnd.nexidia.realtimegrid+json.

Entity Member	Member		Definition
agentList			List of agents to subscribe to the telephony events.
	tisIds		
		agentId	The unique identifier of the agent.
		hubId	The hub that the agent belongs to.
	appIds		
		NXRTGAgentId	The unique identifier of the agent.
		NXRTGOSLogin	The username the agent used to login into the telephony system.
		NXRTGDomain	The authentication domain of the agent.

Request

Update Agent URI provided by System Info.

```
PUT [Agent URI] HTTP/1.1
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: Success

```
HTTP/1.1 204 No Content
```

Response: Invalid Agent Mapping File

```
HTTP/1.1 400 Bad Request
Content-Type: text/plain
Invalid AgentIdMapping File
```

Sample

```
{
  "agentList": [{
    "tisIds": {
      "agentId": "123456",
      "hubId": "123456"
    },
    "appIds": {
      "NXRTGAgentId": "123456",
      "NXRTGOSLogin": "agent1",
      "NXRTGDomain": "domain1"
    }
  }, {
    "tisIds": {
      "agentId": "123457",
      "hubId": "123457"
    },
    "appIds": {
      "NXRTGAgentId": "123457",
      "NXRTGOSLogin": "agent2",
      "NXRTGDomain": "domain2"
    }
  }
]
```

JSON

Get Agent Information

Retrieves the current agent information. See Set Agent Information for a description of the returned entity.

Request

Agent URI provided by System Info.

```
GET [Agent URI] HTTP/1.1
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: Agent Mapping Is Set

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: No Agent Mapping Set

```
HTTP/1.1 204 No Content
```

Get System Topology

Retrieves the current system topology information. If a service is not running or installed, it will not be included in the response.

Request

System Topology URI `http://127.0.0.1:26002/systemtopology`

```
GET [System Topology URI] HTTP/1.1
```

Response: Success

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
{
  "nodeIdentifiers": [
    "ConfigService@127.0.0.1",
    "GatewayService@127.0.0.1",
    "ScanService@127.0.0.1-54985559-82f1-462f-b302-222bf620219f",
    "TelephonyProxyService@127.0.0.1-b2791471-8ef8-4da4-9f1f-77e080489a91"
  ]
}
```

Response: Failure

```
HTTP/1.1 500 Internal Server Error
```

Set Air Information

Updates the air server(s) in RTG for recording. The **Content-Type** header must be set to `application/vnd.nexidia.realtimegrid+json`.

Entity Member	Member	Definition
airList		List of air servers for recording.
	airLoggerId	Air Logger ID.
	airFQDN	Air Server Fully Qualified Domain Name.

Request

Update Air URI provided by System Info.

```
PUT [Air URI] HTTP/1.1
Content-Type: application/vnd.nexidia.realtimegrid+json
```


Response: Success

```
HTTP/1.1 204 No Content
```

Response: Invalid Air server File

```
HTTP/1.1 400 Bad Request
Content-Type: text/plain
Invalid Air Server File
```

Sample

```
{
  "airList": [
    {
      "airLoggerId": 68,
      "airFQDN": "air-01.myDomain.com"
    },
    {
      "airLoggerId": 63,
      "airFQDN": "air-02.myDomain.com"
    }
  ]
}
```

JSON

Get Air Information

Retrieves the current air server information. See Set Air Information for a description of the returned entity.

Request

Air URI provided by System Info.

```
GET [Air URI] HTTP/1.1
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: Air Server Is Set

```
HTTP/1.1 200 OK
Content-Type: application/vnd.nexidia.realtimegrid+json
```

Response: No Air Server Set

```
HTTP/1.1 204 No Content
```

Messaging Interface

Real-Time Grid sends messages via the Java Message Service (JMS) specification as implemented by the ActiveMQ Artemis message broker (<http://activemq.apache.org/>). ActiveMQ Artemis supports client connections from many different client platforms:

- The Java jars are available as part of the main ActiveMQ Artemis download (<http://activemq.apache.org/download.html>). Java developers may also consider using the Spring framework’s facilities for JMS integration (<http://spring.io/>).
- .NET clients can use the .NET Message Service API (NMS) library available from the ActiveMQ site (<http://activemq.apache.org/nms/>).
- Client libraries for other platforms can be found on the ActiveMQ site (<http://activemq.apache.org/connectivity.html>), but these have not been tested with Real-Time Grid.

Destinations

All JMS destinations sent by Real-Time Grid can carry multiple message types, and additional message types may be added in future releases. Therefore, client applications should inspect the JMSType header of each received message to establish how to correctly parse the body.

Destination	Description
Nexidia.RTG.Telephony	A topic for telephony event messages. Two messages (agent/customer) are emitted for every call event on a monitored source.
Nexidia.RTG.Notifications	A topic for notifications of non-telephony events detected during a call. Frequency is influenced by the number and kind of event definitions in the configuration.
Nexidia.RTG.Updates	A topic for high-frequency enlighten score updates of a given call. By default, updates are configured to be published every 10 seconds per call, but can be configured to publish at a different rate.
Nexidia.RTG.Transcript	A topic for ASR transcripts generated for each speaker during a call. By default, transcript messages are disabled and, when enabled, configured to publish every 10 seconds per call.
Nexidia.RTG.Transcript.Finalized	A topic for ASR finalized transcripts generated for each speaker during a call. By default, transcript messages are disabled and, when enabled, configured to publish every 10 seconds per call.

Standard Message Properties

Property	Description
NXRTGSourceId	Identifies the Real-Time Grid service that originated the message. The value format is unspecified may change in subsequent Real-Time Grid releases; however, it is guaranteed not to be longer than 128 characters.
NXRTGSourceTime	Specifies the time that the Real-Time Grid service originated the message. Values are formatted as "YYYY-MM-DDTHH24:MI:ss.SSSXXX" where "XXX" indicates an ISO 8601 time zone; an example value is "2014-01-10T13:49:01.034+5:00."
NXRTGAgentId	Identifies the agent participating on the call. This property has the same value as the agentId body member described below. This property can be used in a JMS message selector to receive messages for a single agent or a group of specific agents.
NXRTGDomain	Identifies the authentication domain of the agent participating on the call. This property has the same value as the domain body member described below. This property can be used in a JMS message selector to receive messages for a single agent or a group of specific agents for a given domain.
NXRTGOsLogin	Identifies the login username of the agent on the call. This property has the same value as the osLogin body member described below. This property can be used in a JMS message selector to receive messages for a single agent or a group of specific agents based on their operating system.

Call Event Messages

The following standard fields may be found in a Nexidia.RTG.Telephony message body.

Body Member	Description
agentId	The unique identifier of the agent taking the call. The value is set from the value of the agentId metadata field, or networkId , if agentId is not available.
callId	The unique identifier of the call within Real-Time Grid. The value is set from the value of the callId metadata field, or recordingGUID , if callId is not available.
domain	The authentication domain for the agent on the call.
osLogin	The username the agent on the call used to login into the telephony system.
callState	The identifier for the state of the call for a given callId. Values could be one of the following: "Connected", "OnHold", "Resume", "Ended".

In addition to the above required fields, an optional callType field can be expected in most environments. The callType can be used to filter calls based on call direction. The following two directions are expected in production environments:

- Incoming: Indicates a customer or agent has called an agent
- Outgoing: Indicates an agent has called a customer

The callType will be populated with the same value in the call signaling for all segments of a call, i.e. if the agent transfers an Outgoing call to another agent, the callType will still be Outgoing.

CallStart

Sent after Real-Time Grid receives a Call Start notification via HTTP from the Telephony Proxy Service.

JMSType	Nexidia.RTG.CallStart
Destination	Nexidia.RTG.Telephony

Example Message Body

```

{
  "agentId": "87914",
  "bd_SkillGroupNumber": "461109",
  "agentPhoneExtension": "521412",
  "rtMonitorMetadata": "5ca7889c-822c-4489-8b55-e479023ecfc6",
  "bd_nvcExportRule": "",
  "callType": "Incoming",
  "bd_CurrentCalledName": "",
  "callingParty": "9491234567",
  "bd_CalledName": "",
  "empName": "Mary Jones",
  "bd_ANI": "9491234567",
  "calledParty": "521412",
  "endRecordEvent": null,
  "callId": "6850138667696078719",
  "streamUrl": "server.company.com",
  "callStartTime": "0001-01-01T00:00:00",
  "audioCodec": "G.729",
  "callState": "Connected",
  "bd_DNIS": "521412",
  "bd_iEtkExported": "-1",
  "bd_CurrentCallingName": "9491234567 ",
  "bd_MIC_PhoneNumber": "",
  "endCallEvent": null,
  "startCallRecEvent": {
    "ClsEventTime": "2020-07-16T11:56:37.3761795-06:00",
    "EventType": 405,
    "InteractionInfo": {
      "BusinessData": {
        "CurrentCallingName": "9491234567 ",
        "nvcExportRule": "",
        "DNIS": "521412",
        "iEtkExported": "-1",
        "CurrentCalledName": "",
        "SkillGroupNumber": "461109",
        "ANI": "9491234567",
        "extensiondata": "",
        "MIC_PhoneNumber": "",
        "CalledName": ""
      },
      "UniversalCallID": "",
      "InteractionID": 6850138667696078719,
      "ContactStartTime": "2020-07-16T11:56:36.76-06:00",
      "StartTime": "2020-07-16T11:56:36.76-06:00",
      "CompoundID": 6850138667696078719,
      "PBXCallID": 192148379,
      "ExternalParticipantInfoArray": [{
        "DialedIn": "",
        "TrunkNumber": "",
        "IsFirstUser": false,
        "TrunkGroup": "",
        "IsInteractionInitiator": true,
        "ClientID": "",
        "PhoneNumber": "9491234567",
        "RecordingStatus": 1,
        "TrunkLabel": "",
        "RecordingMediaList": [{
          "LoggerId": 126,
          "MMLRecordingHint": "",
          "InteractionID": 6850138667696078719,
          "RTMonitorMetaData": "5ca7889c-822c-4489-8b55-e479023ecfc6",
          "Channel": "-1"
        }]
      }]
    }
  }
}

```

```

        "StartTime": "2020-07-16T11:56:37.1991353-06:00",
        "TimeDiff": "00:00:00.2804929",
        "RecProgram": "95",
        "RecordingType": 1,
        "InitiatorType": 48,
        "RecordingSideTypeId": 1,
        "SessionID": 6850138676049020788,
        "Summation": 1,
        "StopTime": "0001-01-01T00:00:00"
    }
}
],
"DirectionType": 1,
"InternalParticipantInfoArray": [{
    "ObjectId": 0,
    "DeviceId": 0,
    "RecordingStatus": 1,
    "SwitchId": 2,
    "Station": "521412",
    "DeviceType": "Station",
    "Department": "",
    "UserId": 87914,
    "IsInteractionInitiator": false,
    "PhoneNumber": "",
    "AgentId": "51412",
    "AgentName": "",
    "RecordingMediaList": [{
        "LoggerId": 126,
        "MMLRecordingHint": "",
        "InteractionID": 6850138667696078719,
        "RTMonitorMetaData": "5ca7889c-822c-4489-8b55-e479023ecfc6",
        "Channel": -1,
        "StartTime": "2020-07-16T11:56:37.1991353-06:00",
        "TimeDiff": "00:00:00.2804929",
        "RecProgram": "95",
        "RecordingType": 1,
        "InitiatorType": 48,
        "RecordingSideTypeId": 2,
        "SessionID": 6850138676049020788,
        "Summation": 2,
        "StopTime": "0001-01-01T00:00:00"
    }
}
]
},
"ClientDTMF": "",
"PreventedMedia": 0,
"HoldMedia": 0,
"PBXCallIndex": 0,
"CurrentRecordingMedia": 1,
"VectorNumber": "",
"ExternalCallID": "",
"dialedIn": "521412"
},
"ClsSiteId": 1,
"EventID": 137982,
"SentByPushTimeStamp": "2020-07-16T11:56:37.1509858-06:00",
"RequestsArrayList": [],
"SubscriptionIDs": [1810968041],
"CurrentRecordingStatus": 1,
"MonitorTimeStamp": "2020-07-16T11:56:37.1509858-06:00"

```

```
    },  
    "bd_extensiondata": ""  
  }  
}
```

CallPause

Sent after Real-Time Grid receives a Call OnHold notification via HTTP from the Telephony Proxy Service.

JMSType	Nexidia.RTG.CallPause
Destination	Nexidia.RTG.Telephony::Nexidia.RTG.Telephony

Example Message Body

```

{
  "agentId": "111025",
  "bd_SkillGroupNumber": "450531",
  "agentPhoneExtension": "521508",
  "rtMonitorMetadata": "55b63827-96a4-43b5-baca-e033154e8ec0",
  "bd_nvcExportRule": "",
  "callType": "Incoming",
  "bd_CurrentCalledName": "",
  "callingParty": "4431234567",
  "bd_CalledName": "",
  "empName": "James Smith",
  "bd_ANI": "4431234567",
  "calledParty": "521508",
  "endRecordEvent": {
    "ClsEventTime": "2020-07-16T11:50:15.1125044-06:00",
    "EventType": 406,
    "InteractionInfo": {
      "BusinessData": {
        "CurrentCallingName": "4431234567 ",
        "nvcExportRule": "",
        "DNIS": "521508",
        "iEtkExported": "-1",
        "CurrentCalledName": "",
        "SkillGroupNumber": "450531",
        "ANI": "4431234567",
        "extensiondata": "",
        "MIC_PhoneNumber": "",
        "CalledName": ""
      },
      "UniversalCallID": "",
      "InteractionID": 6850136885284650241,
      "ContactStartTime": "2020-07-16T11:49:51.07-06:00",
      "StartTime": "2020-07-16T11:49:51.07-06:00",
      "CompoundID": 6850136885284650241,
      "PBXCallID": 183331231,
      "ExternalParticipantInfoArray": [{
        "DialedIn": "",
        "TrunkNumber": "",
        "IsFirstUser": false,
        "TrunkGroup": "",
        "IsInteractionInitiator": true,
        "ClientID": "",
        "PhoneNumber": "4431234567",
        "RecordingStatus": 0,
        "TrunkLabel": "",
        "RecordingMediaList": [{
          "LoggerId": 126,
          "MMLRecordingHint": "",
          "InteractionID": 6850136885284650241,
          "RTMonitorMetaData": "55b63827-96a4-43b5-baca-e033154e8ec0",
          "Channel": -1,
          "StartTime": "2020-07-16T11:49:51.4901145-06:00",
          "TimeDiff": "00:00:00.2688898",
          "RecProgram": "95",
          "RecordingType": 1,
          "InitiatorType": 48,
          "RecordingSideTypeId": 1,
          "SessionID": 6850136932292298508,
          "Summation": 1,
          "StopTime": "2020-07-16T11:50:15.1065038-06:00"
        }
      ]
    }
  }
}

```



```

    }
  ],
  "DirectionType": 1,
  "InternalParticipantInfoArray": [{
    "ObjectId": 0,
    "DeviceId": 0,
    "RecordingStatus": 0,
    "SwitchId": 2,
    "Station": "521508",
    "DeviceType": "Station",
    "Department": "",
    "UserId": 111025,
    "IsInteractionInitiator": false,
    "PhoneNumber": "",
    "AgentId": "51508",
    "AgentName": "",
    "RecordingMediaList": [{
      "LoggerId": 126,
      "MMLRecordingHint": "",
      "InteractionID": 6850136885284650241,
      "RTMonitorMetaData": "55b63827-96a4-43b5-baca-e033154e8ec0",
      "Channel": -1,
      "StartTime": "2020-07-16T11:49:51.4901145-06:00",
      "TimeDiff": "00:00:00.2688898",
      "RecProgram": "95",
      "RecordingType": 1,
      "InitiatorType": 48,
      "RecordingSideTypeId": 2,
      "SessionID": 6850136932292298508,
      "Summation": 2,
      "StopTime": "2020-07-16T11:50:15.1065038-06:00"
    }
  ]
}

},
"ClientDTMF": "",
"PreventedMedia": 0,
"HoldMedia": 1,
"PBXCallIndex": 0,
"CurrentRecordingMedia": 0,
"VectorNumber": "",
"ExternalCallID": "",
"dialedIn": "521508"
},
"ClsSiteId": 1,
"EventID": 137833,
"SentByPushTimeStamp": "2020-07-16T11:50:14.8846135-06:00",
"RequestsArrayList": [],
"SubscriptionIDs": [1810968040],
"CurrentRecordingStatus": 0,
"MonitorTimeStamp": "2020-07-16T11:50:14.8846135-06:00"
},
"callId": "6850136885284650241",
"streamUrl": "server.company.com",
"callStartTime": "0001-01-01T00:00:00",
"audioCodec": "G.729",
"callState": "OnHold",
"bd_DNIS": "521508",
"bd_iEtkExported": "-1",
"bd_CurrentCallingName": "4431234567 ",
"bd_MIC_PhoneNumber": "",
"endCallEvent": null

```

```
"startCallRecEvent": null,  
"bd_extensiondata": ""  
}
```

CallResume

Sent after Real-Time Grid receives a Call Resume notification via HTTP from the Telephony Proxy Service.

JMSType	Nexidia.RTG.CallResume
Destination	Nexidia.RTG.Telephony:Nexidia.RTG.Telephony

Example Message Body

```
{
  "agentId": "6",
  "callId": "6689407874305688247",
  "domain": "FAST-TALK",
  "osLogin": "user2009",
  "callState": "Resume",
  "streamUrl": "AIR-WS-2012-r2",
  "callStartTime": "0001-01-01T00:00:00",
  "agentPhoneExtension": "2009",
  "audioCodec": "G.729",
  "rtMonitorMetadata": "d06150e1-3ba7-40f7-9d6f-b0acb22287a0",
  "callType": "Outgoing",
  "callingParty": "2009",
  "empName": "2009 Agent",
  "endCallEvent": null,
  "calledParty": "97709100576",
  "endRecordEvent": null,
  "startCallRecEvent": {
    "ClsEventTime": "2019-05-10T14:40:14.9413843-04:00",
    "EventType": 405,
    "InteractionInfo": {
      "BusinessData": {},
      "UniversalCallID": "",
      "InteractionID": 6689407874305688247,
      "ContactStartTime": "2019-05-10T10:39:01.933-04:00",
      "StartTime": "2019-05-10T10:39:01.933-04:00",
      "CompoundID": 6689407848535884470,
      "PBXCallID": 16846339,
      "ExternalParticipantInfoArray": [{
        "DialedIn": "",
        "TrunkNumber": "",
        "IsFirstUser": false,
        "TrunkGroup": "",
        "IsInteractionInitiator": false,
        "ClientID": "",
        "PhoneNumber": "97709100576",
        "RecordingStatus": 1,
        "TrunkLabel": "",
        "RecordingMediaList": [{
          "LoggerId": 63,
          "MMLRecordingHint": "",
          "InteractionID": 6689407874305688247,
          "RTMonitorMetaData": "d06150e1-3ba7-40f7-9d6f-b0acb22287a0",
          "Channel": -1,
          "StartTime": "2019-05-10T10:40:14.9343784-04:00",
          "TimeDiff": "00:00:00.2236033",
          "RecProgram": "",
          "RecordingType": 1,
          "InitiatorType": 32,
          "RecordingSideTypeId": 1,
          "SessionID": 6689407878575489180,
          "Summation": 1,
          "StopTime": "0001-01-01T00:00:00"
        }, {
          "LoggerId": 63,
          "MMLRecordingHint": "",
          "InteractionID": 6689407874305688247,
          "RTMonitorMetaData": "d06150e1-3ba7-40f7-9d6f-b0acb22287a0",
          "Channel": -1,
          "StartTime": "2019-05-10T10:39:02.1778583-04:00",
          "TimeDiff": "00:00:00.2236033",
          "RecProgram": ""
        }
      ]
    }
  }
}
```

```

"RecordingType": 1,
"InitiatorType": 32,
"RecordingSideTypeId": 1,
"SessionID": 6689407878575489180,
"Summation": 1,
"StopTime": "2019-05-10T10:39:30.0088803-04:00"
}
]
},
"DirectionType": 2,
"InternalParticipantInfoArray": [{
"ObjectId": 0,
"DeviceId": 0,
"RecordingStatus": 1,
"SwitchId": 1,
"Station": "2009",
"DeviceType": "Station",
"Department": "",
"UserId": 6,
"IsInteractionInitiator": true,
"PhoneNumber": "",
"AgentId": "",
"AgentName": "",
"RecordingMediaList": [{
"LoggerId": 63,
"MMLRecordingHint": "",
"InteractionID": 6689407874305688247,
"RTMonitorMetaData": "d06150e1-3ba7-40f7-9d6f-b0acb22287a0",
"Channel": -1,
"StartTime": "2019-05-10T10:40:14.9343784-04:00",
"TimeDiff": "00:00:00.2236033",
"RecProgram": "",
"RecordingType": 1,
"InitiatorType": 32,
"RecordingSideTypeId": 2,
"SessionID": 6689407878575489180,
"Summation": 2,
"StopTime": "0001-01-01T00:00:00"
}, {
"LoggerId": 63,
"MMLRecordingHint": "",
"InteractionID": 6689407874305688247,
"RTMonitorMetaData": "d06150e1-3ba7-40f7-9d6f-b0acb22287a0",
"Channel": -1,
"StartTime": "2019-05-10T10:39:02.1778583-04:00",
"TimeDiff": "00:00:00.2236033",
"RecProgram": "",
"RecordingType": 1,
"InitiatorType": 32,
"RecordingSideTypeId": 2,
"SessionID": 6689407878575489180,
"Summation": 2,
"StopTime": "2019-05-10T10:39:30.0088803-04:00"
}
]
},
"ClientDTMF": "",
"PreventedMedia": 0,
"HoldMedia": 0,
"PBXCallIndex": 0

```

```

"CurrentRecordingMedia": 1,
"VectorNumber": "",
"ExternalCallID": "",
"dialedIn": "97709100576"
},
"ClsSiteId": 1,
"EventID": 100425,
"SentByPushTimeStamp": "2019-05-10T10:40:15.0745189-04:00",
"RequestsArrayList": [],
"SubscriptionIDs": [1490808401],
"CurrentRecordingStatus": 1,
"MonitorTimeStamp": "2019-05-10T10:40:14.9453885-04:00"
}
}

```

CallEnd

Indicates that a call has ended. Sent after Real-Time Grid receives a Call Ended notification via HTTP from the Telephony Proxy Service.

In exceptional situations Real-Time Grid may *terminate* a call that has experienced an irrecoverable error. In such cases the message will contain an **error** member with the code **CallTerminated** and a message describing the nature of the error.

Terminated calls may lack metadata fields that are normally available in CallEnd messages because RTG did not receive a normal Call End notification. Also, some metadata values may not have updated; for example, **callState** may still be set to "Connected."

JMSType	Nexidia.RTG.CallEnd
Destination	Nexidia.RTG.Telephony::Nexidia.RTG.Telephony

Example Message Body

```

{
  "agentId": "13820",
  "bd_SkillGroupNumber": "461126",
  "agentPhoneExtension": "521428",
  "rtMonitorMetadata": "c885d1c1-9ae3-4356-9d74-c0a05268cb0a",
  "bd_nvcExportRule": "",
  "callType": "Incoming",
  "bd_CurrentCalledName": "",
  "callingParty": "3471234567",
  "bd_CalledName": "",
  "empName": "Paul Stevens",
  "bd_ANI": "3471234567",
  "calledParty": "521428",
  "endRecordEvent": null,
  "callId": "6850083013509856306",
  "streamUrl": "server.company.com",
  "callStartTime": "0001-01-01T00:00:00",
  "audioCodec": "G.729",
  "callState": "Ended",
  "bd_DNIS": "521428",
  "bd_iEtkExported": "-1",
  "bd_CurrentCallingName": "3471234567 ",
  "bd_MIC_PhoneNumber": "",
  "endCallEvent": {
    "ClsEventTime": "2020-07-16T08:20:45.3084818-06:00",
    "EventType": 404,
    "InteractionInfo": {
      "BusinessData": {
        "CurrentCallingName": "3471234567 ",
        "nvcExportRule": "",
        "DNIS": "521428",
        "iEtkExported": "-1",
        "CurrentCalledName": "",
        "SkillGroupNumber": "461126",
        "ANI": "3471234567",
        "extensiondata": "",
        "MIC_PhoneNumber": "",
        "CalledName": ""
      },
      "UniversalCallID": "",
      "InteractionID": 6850083013509856306,
      "ContactStartTime": "2020-07-16T08:20:38.187-06:00",
      "StartTime": "2020-07-16T08:20:38.187-06:00",
      "CompoundID": 6850083013509856306,
      "PBXCallID": 229941465,
      "ExternalParticipantInfoArray": [{
        "DialedIn": "",
        "TrunkNumber": "",
        "IsFirstUser": false,
        "TrunkGroup": "",
        "IsInteractionInitiator": true,
        "ClientID": "",
        "PhoneNumber": "3471234567",
        "RecordingStatus": 1,
        "TrunkLabel": "",
        "RecordingMediaList": [{
          "LoggerId": 126,
          "MMLRecordingHint": "",
          "InteractionID": 6850083013509856306,
          "RTMonitorMetaData": "c885d1c1-9ae3-4356-9d74-c0a05268cb0a",
          "Channel": -1,
          "StartTime": "2020-07-16T08:20:38.4276359-06:00"
        }]
      }]
    }
  }
}

```

```

        "TimeDiff": "00:00:00.0722960",
        "RecProgram": "95",
        "RecordingType": 1,
        "InitiatorType": 48,
        "RecordingSideTypeId": 1,
        "SessionID": 6850083017567845784,
        "Summation": 1,
        "StopTime": "2020-07-16T08:20:45.217-06:00"
    }
}
],
"DirectionType": 1,
"InternalParticipantInfoArray": [{
    "ObjectId": 0,
    "DeviceId": 0,
    "RecordingStatus": 1,
    "SwitchId": 2,
    "Station": "521428",
    "DeviceType": "Station",
    "Department": "",
    "UserId": 13820,
    "IsInteractionInitiator": false,
    "PhoneNumber": "",
    "AgentId": "51428",
    "AgentName": "",
    "RecordingMediaList": [{
        "LoggerId": 126,
        "MMLRecordingHint": "",
        "InteractionID": 6850083013509856306,
        "RTMonitorMetaData": "c885d1c1-9ae3-4356-9d74-c0a05268cb0a",
        "Channel": -1,
        "StartTime": "2020-07-16T08:20:38.4276359-06:00",
        "TimeDiff": "00:00:00.0722960",
        "RecProgram": "95",
        "RecordingType": 1,
        "InitiatorType": 48,
        "RecordingSideTypeId": 2,
        "SessionID": 6850083017567845784,
        "Summation": 2,
        "StopTime": "2020-07-16T08:20:45.217-06:00"
    }
}
]
},
"ClientDTMF": "",
"PreventedMedia": 0,
"HoldMedia": 0,
"PBXCallIndex": 0,
"CurrentRecordingMedia": 1,
"VectorNumber": "",
"ExternalCallID": "",
"StopTime": "2020-07-16T08:20:45.217-06:00",
"dialedIn": "521428"
},
"ClsSiteId": 1,
"IsCompoundEnded": true,
"EventID": 136033,
"SentByPushTimeStamp": "2020-07-16T08:20:45.3827375-06:00",
"RequestsArrayLIst": [],
"SubscriptionIDs": [1810968033],
"CurrentRecordingStatus": 1

```

```

    "MonitorTimeStamp": "2020-07-16T08:20:45.3827375-06:00"
  },
  "startCallRecEvent": null,
  "bd_extensiondata": ""
}

```

Telephony Proxy metadata filtering

Filters define calls to be diverted from scanning. Filters use the same Any/All, IsIn/IsNotIn logic that the rules engine uses. Rules can be created for any CTI/business metadata field. If the specified key does not exist in the metadata a warning will be logged.

In this sample configuration, the operator is "Any" so if either of these two conditions are met the call will not be sent to the scan process:

- agentPhoneExtension 2009 or 2016
- domain is FAST-TALK

Sample file:

```

{
  "configurationId": "simple_phrase-1",
  "metadataFilters": {
    "operator": "Any",
    "filters": [{
      "key": "agentPhoneExtension",
      "operator": "IsNotIn",
      "values": [
        "2009",
        "2016"
      ]
    }, {
      "key": "domain",
      "operator": "IsIn",
      "values": [
        "FAST-TALK"
      ]
    }
  ]
},
  "eventDefinitions": [{
    ...
  }
]
}

```

JSON

Event Notification Message

Indicates that an event was detected in the call. The **notificationType** field is set to "Primary" or "Secondary".

The **detail** field, if present, contains information about the event to help explain how the event was detected. Although the field is human-readable, it should not be presented to end users. The format of `detail` values is unspecified, and it will change in future releases.

The following standard fields may be found in a `Nexidia.RTG.Notifications` message body.

Body Member	Description
agentId	Unique identifier of the agent taking the call. The value is set from the value of the agentId metadata field, or networkId , if agentId is not available.
callId	Unique identifier of the call within Real-Time Grid. The value is set from the value of the callId metadata field, or recordingGUID , if callId is not available.
domain	Authentication domain for the agent on the call.
osLogin	Username the agent on the call used to login into the telephony system.
configurationId	Identifier for the configuration applied to this call. If the client application did not specify a value for configurationId this field is omitted.
workflowGuid	GUID identifying the Workflow used to scan the call.
notificationId	GUID identifying a notification message
notificationType	Type of this notification message. Values are either "Primary" or "Secondary".
eventId	Unique integer identifier for an Event definition as defined in the configuration.json
offsetMs	Milliseconds from the call start to when the event was triggered.
notificationCount	Number of times a configured Event has been triggered and a primary notification message has been sent. Applies to discrete Events or to the first occurrence of a continuous Event.
secondaryNotificationCount	Number of times a configured secondary notification has been sent for a continuous Event. Applies only to continuous Events.
stream	String value of either "Agent" or "Other" if the call is stereo, or "Either" if the call is mono. "Other" specifies the customer channel.
matchedPhrases	JSON array of configured phrases discovered during the call. These phrases are set as phrase Rules in the configuration.json.
enlightenResults	JSON array. Each JSON member includes an enlightenModelGuid string value, a double floating point score for that particular Enlighten model, and a boolean specifying whether the provided score is valid or should not be considered by the consuming application.

Body Member	Description
notificationPayload	Pass-through JSON object opaque to RTG. It is intended to be an application specific payload set in the configuration and forwarded along in the notification message when the Event is triggered. The fields shown in the example below are examples and should not be taken to have any particular meaning.
callMetadata	Pass-through JSON object containing fields specific to a telephony environment. Fields in callMetadata are not guaranteed to be available or reliable for the consuming application.

JMSType	Nexidia.RTG.EventNotification
Destination	Nexidia.RTG.Notifications

Example Message Body

```
{
  "agentId": "1234",
  "callId": "70d0dfb5-9724-4130-b5c3-6cbc4e484fb5",
  "domain": "FAST-TALK",
  "osLogin": "user2009",
  "configurationId": "simple_phrase-1",
  "notificationId": "70f7c4ce-1c9b-4e7a-868f-53d48f917632",
  "notificationType": "Primary",
  "eventId": 7,
  "offsetMs": 32290,
  "notificationCount": 1,
  "secondaryNotificationCount": 0,
  "stream": "Either",
  "workflowGuid": "2360bd2d-6b69-4b2c-b672-9b272be0f2b2",
  "nxrtgAgentId": "testagent1",
  "nxrtgCallId": "94277b8f-0153-45cd-8295-ce2beaf8dc69.05212005-100",
  "matchedPhrases": ["modem"],
  "enlightenResults": [],
  "callMetadata": {
    "agentAcidLoginId": "1001",
    "agentId": "testagent1",
    "agentIpAddress": null,
    "agentPhoneExtension": "1001",
    "audioCodec": "G.711",
    "callId": "94277b8f-0153-45cd-8295-ce2beaf8dc69.05212005-100",
    "calledParty": "1001",
    "calledPartyName": null,
    "callingParty": "4044957251",
    "callingPartyName": null,
    "callStartTime": "Mon Aug 15 10:58:31 EDT 2022",
    "callState": "Connected",
    "callType": "Inbound",
    "empCode": "70001",
    "empName": "Test Agent1",
    "globalCallId": null,
    "networkId": "testagent1",
    "pbxCallId": "05212005100",
    "recordingGUID": "94277b8f-0153-45cd-8295-ce2beaf8dc69.05212005-100",
    "skillGroupId": "100",
    "skillGroupName": "Billing",
    "streamUrl": "rtsp://10.1.15.31:554/94277b8f-0153-45cd-8295-ce2beaf8dc69.05212005-100",
    "voiceServerIp": null
  },
  "notificationPayload": {}
}
```

Enlighten Update Message

Indicates that one or more Enlighten models have changed the evaluation of the call.

The following standard fields may be found in a `Nexidia.RTG.Updates` message body.

Body Member	Description
agentId	The unique identifier of the agent taking the call. The value is set from the value of the agentId metadata field, or networkId , if agentId is not available.

Body Member	Description
callId	The unique identifier of the call within Real-Time Grid. The value is set from the value of the callId metadata field, or recordingGUID , if callId is not available.
domain	The authentication domain for the agent on the call.
osLogin	The username the agent on the call used to login into the telephony system.
configurationId	The identifier for the configuration being applied to this call. If the client application did not specify a value for configurationId then this field is omitted.
workflowGuid	The GUID identifying the Workflow model used to scan the call.
offsetMs	A 64 bit integer value corresponding to the number of milliseconds past the start of the call when the scores have been updated.
enlightenResults	A json array of objects that each contain an "enlightenModelGuid" string, a 64 bit floating point "score" value corresponding to that model, and a "isValid" boolean value indicating whether the provided score is valid or if it should not be used by the consuming application.

TIP

These messages can be sent frequently, up to one per second per call.

JMSType	Nexidia.RTG.EnlightenUpdate
Destination	Nexidia.RTG.Updates::Nexidia.RTG.Updates

Example Message Body

```

{
  "configurationId": "3",
  "agentId": "60502",
  "callId": "8a56739c-f50b-4321-b17c-c75ed8f7c320",
  "domain": "",
  "osLogin": "",
  "workflowGuid": "2360bd2d-6b69-4b2c-b672-9b272be0f2b2",
  "offsetMs": 230,
  "enlightenResults": [{
    "score": 0.46868587,
    "enlightenModelGuid": "0f9142bb-0f6a-4786-a20d-015f6b1190e9",
    "isValid": false,
    "enlightenModelTag": "AcknowledgeLoyalty"
  }, {
    "score": 0.059134427,
    "enlightenModelGuid": "057aceb5-300a-464e-9ec4-83ec64aea3e6",
    "isValid": false,
    "enlightenModelTag": "ActiveListening"
  }, {
    "score": 0.010129569,
    "enlightenModelGuid": "6e02a01f-a61a-40d6-9da0-26ec1a63c4a8",
    "isValid": false,
    "enlightenModelTag": "BeEmpathetic"
  }, {
    "score": 0.02682019,
    "enlightenModelGuid": "6281b662-1823-4866-9b17-1a3f1e3feedb",
    "isValid": false,
    "enlightenModelTag": "BuildRapport"
  }, {
    "score": 0.072687425,
    "enlightenModelGuid": "d14dc42e-e72b-4ed8-8cf5-dfeb37050084",
    "isValid": false,
    "enlightenModelTag": "DemonstrateOwnership"
  }, {
    "score": 0.030276816,
    "enlightenModelGuid": "c1dd74d9-4314-4acd-9700-98dbcaeb6553",
    "isValid": false,
    "enlightenModelTag": "EffectiveQuestioning"
  }, {
    "score": 0.35011122,
    "enlightenModelGuid": "7b4534e8-181e-45af-9f69-72b95a83fab8",
    "isValid": false,
    "enlightenModelTag": "InappropriateAction"
  }, {
    "score": 1.0,
    "enlightenModelGuid": "a0a6ceef-6b40-4d8c-8473-0cb7a8b67389",
    "isValid": true,
    "enlightenModelTag": "Interrupton"
  }, {
    "score": 0.5817162,
    "enlightenModelGuid": "bc614023-d5c5-4642-aed3-d62eafda5a78",
    "isValid": false,
    "enlightenModelTag": "PromoteSelfService"
  }, {
    "score": 0.13467652,
    "enlightenModelGuid": "58cd2983-32ba-4d4c-af62-228581b04b1a",
    "isValid": false,
    "enlightenModelTag": "Sentiment"
  }, {
    "score": 0.32163802,
    "enlightenModelGuid": "ce29b5b7-cc95-432d-8548-ad9683db0677",
    "isValid": false,

```

https://wem-dochub.mindtouch.us/Nexidia_Analytics_and_Data_Exchange_Framework/NXQC_12.5/Grid_Documentati...

Updated: Thu, 30 Apr 2026 18:13:28 GMT

```
    "enlightenModelTag": "SetExpectations"
  }, {
    "score": 0.0,
    "enlightenModelGuid": "142b1843-fd24-4718-a074-067984dbad63",
    "isValid": false,
    "enlightenModelTag": "SpeechVelocity"
  }
]
```

Transcript Message

A running ASR transcript of all words said on one leg of a call. The transcript updates periodically. The transcript update includes an identifier for the configuration and Workflow that produced the results as well as the agent identifier for the agent being monitored for the call.

Each token (word or redacted hashes) is presented with its offset and an identifier, Agent or Other, for which leg of the call produced the token.

The following standard fields may be found in a `Nexidia.RTG.Transcript` and `Nexidia.RTG.Transcript.Finalized` message body.

Body Member	Description
agentId	The unique identifier of the agent taking the call. The value is set from the value of the agentId metadata field, or networkId , if agentId is not available.
callId	The unique identifier of the call within Real-Time Grid. The value is set from the value of the callId metadata field, or recordingGUID , if callId is not available.
domain	The authentication domain for the agent on the call.
osLogin	The username the agent on the call used to login into the telephony system.
configurationId	The identifier for the configuration being applied to this call. If the client application did not specify a value for configurationId this field is omitted.
workflowGuid	The GUID identifying the Workflow used to scan the call.
transcript	JSON array of tokens, each of which represents a word. Each entry includes an ASR string for the token, the stream identifying the leg of the call on which the token was found, and a score indicating the likelihood the token was spoken. Valid stream values are AGENT, CUSTOMER, UNKNOWN, ANY, or SYSTEM. Stereo calls will specify the stream as AGENT or CUSTOMER. The other values are for mono calls.

TIP

Transcript messages may be sent frequently, up to one per second per call.

JMSType	Nexidia.RTG.Transcript
---------	------------------------

Destination	Nexidia.RTG.Transcript
-------------	------------------------

Example Message Body

```
{
  "callId": "ccf10bd5-1882-436e-84b9-648403641847",
  "agentId": "80003",
  "domain": "FAST-TALK",
  "osLogin": "user2009",
  "configurationId": "fires alert whenever 'hello' is said",
  "workflowGuid": "2360bd2d-6b69-4b2c-b672-9b272be0f2b2",
  "transcript": [
    {
      "token": "HELLO",
      "stream": "CUSTOMER",
      "offsetMs": 650,
      "endOffsetMs": 750,
      "score": 1.0
    },
    {
      "token": "THIS",
      "stream": "CUSTOMER",
      "offsetMs": 820,
      "endOffsetMs": 900,
      "score": 1.0
    },
    {
      "token": "IS",
      "stream": "CUSTOMER",
      "offsetMs": 910,
      "endOffsetMs": 990,
      "score": 0.99990845
    }
  ]
}
```

JSON

JMSType	Nexidia.RTG.Transcript.Finalized
Destination	Nexidia.RTG.Transcript.Finalized

Example Message Body

```
{
  "callId": "ccf10bd5-1882-436e-84b9-648403641847",
  "agentId": "80003",
  "domain": "FAST-TALK",
  "osLogin": "user2009",
  "configurationId": "fires alert whenever 'hello' is said",
  "workflowGuid": "2360bd2d-6b69-4b2c-b672-9b272be0f2b2",
  "transcript": [
    {
      "token": "HELLO",
      "stream": "CUSTOMER",
      "offsetMs": 650,
      "endOffsetMs": 750,
      "score": 1.0,
      "finalized": true
    },
    {
      "token": "THIS",
      "stream": "CUSTOMER",
      "offsetMs": 820,
      "endOffsetMs": 900,
      "score": 1.0,
      "finalized": true
    },
    {
      "token": "IS",
      "stream": "CUSTOMER",
      "offsetMs": 910,
      "endOffsetMs": 990,
      "score": 0.99990845,
      "finalized": true
    }
  ]
}
```

- Processing and network latencies can cause a EnlightenUpdate message to be delivered after the CallEnd message for the corresponding call.
- The configurationId member corresponds to the id member of the corresponding configuration. See Set Configuration for details.

Nexdia Interaction Analytics (NIA) Integration

The **notificationPayload** member supports allows application-specific information to be passed between a component that sets the configuration and the component that presents or otherwise processes an Event notification message. The Nexidia Scan product uses the **notificationPayload** to coordinate between its Administration application and the two alert presentation applications, Agent Desktop and Observer Console. In some deployment scenarios a user of Real-Time Grid may use Nexidia Scan's Administration system, but process the Event notifications in a custom application. To do so requires an understanding of the fields that Nexidia Scan places in the **notificationPayload** field.

In the context of Nexidia Scan's **notificationPayload** fields several words have specific meanings that require explanation:

- Alert: presentation of an Event notification message to an end user.
- Primary Alert: alert corresponding to a discrete event or to the first presentation of a continuous Event for a call.
- Secondary Alert: alert corresponding to subsequent presentations of a continuous Event. The timing of secondary alerts is governed by the **secondaryTimeBasisMs** field in the configuration.

A single notification payload may contain fields referring to both primary and secondary alerts. The presentation application uses the **notificationType** value to decide which sets values to use.

Notification Payload

The Notification Payload is specified by NIA or any other client used to configure RTG Events. The payload is opaque to RTG. Fields defined below are examples, subject to change by NIA, and are not controlled by RTG.

The following table describes fields Nexidia Scan places in the notification payload.

Fields	Description
isOmission	Boolean that is true if the Event uses the operator values of "NotAny" or "NotAll" and false otherwise.
hasPhraseExpression	Boolean that is true if the Event involves a phrase expression, whether it is for spoken phrases or for omissions.
hasSentiment	Boolean that is true if the Event involves sentiment detection. Note that sentiment detection is the only type of Enlighten that Nexidia Scan uses.
activeFrom	If the current time is before the value of this field, the alert should not be presented. Omitted if there is no such constraint. The value is formatted based on ISO 8601 with a maximum length of 32 characters.
activeTo	If the current time is after the value of this field, the alert should not be presented. Omitted if there is no such constraint. The value is formatted based on ISO 8601 with a maximum length of 32 characters.
primaryAlertNam	The name to present in the context of a primary alert. Maximum length 50 characters.
primaryAlertDescription	A short description to present in the context of a primary alert. Maximum length 1000 characters.

Fields	Description
isPrimaryAlertForAgent	True if, in the context of a primary alert, this alert should be presented to an agent user.
isPrimaryAlertForSupervisor	True if, in the context of a primary alert, this alert should be presented to an supervisor user.
primaryAlertArticleUrl	Contains the URL of an article relevant to the alert. This field is optional, and is omitted if an article is not set. Maximum length 200 characters. The URL points to the Nexidia Scan document server, which will return the document itself (with response code 200) or redirect to a different server (with response code 302).
secondaryAlertName	The name to present in the context of a secondary alert. Maximum length 50 characters.
secondaryAlertDescription	A short description to present in the context of a secondary alert. Maximum length 1000 characters.
isSecondaryAlertForAgent	True if, in the context of a secondary alert, this should be presented to an agent user.
isSecondaryAlertForSupervisor	True if, in the context of a secondary alert, this should be presented to an supervisor user.
secondaryAlertArticleUrl	Same structure and semantics as the primaryAlertArticleUrl member, but applies only in the context of a secondary alert. Maximum length 200 characters.

Example Notification Payload

```
notificationPayload: {
  "isOmission": true,
  "hasPhraseExpression": true,
  "hasEnlightenModel": false,
  "activeFrom": "2014-04-04T03:00:00-04:00",
  "activeTo": "2014-05-29T03:00:00-04:00",
  "primaryAlertName": "Disclaimer",
  "primaryAlertDescription": "Remember to state the disclaimer.",
  "isPrimaryAlertForAgent": true,
  "isPrimaryAlertForSupervisor": false,
  "primaryAlertArticleUrl": "http://server/articles/
disclaimer.doc",
  "secondaryAlertName": "Disclaimer!",
  "secondaryAlertDescription": "Omitting the disclaimer is
risky!",
  "isSecondaryAlertForAgent": true,
  Nexidia Real-Time Grid Programmer's Guide 53
  "isSecondaryAlertForSupervisor": true,
  "secondaryAlertArticleUrl": "http://server/articles/
disclaimer.doc"
}
```

References

- Fielding, R.T. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation. University of California, Irvine. Available from <http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm> (2000); accessed 3 August 2010.
- Josefsson, S. The Base16, Base32, and Base64 Data Encodings. Available from <http://www.ietf.org/rfc/rfc4648.txt> (2006); accessed 25 March 2011.